OPENAI CODEX FULL COURSE 2026

OpenAI Codex Full Course

Skills, MCPs, Subagents, and Everything That Actually Matters

Aniket Panjwani

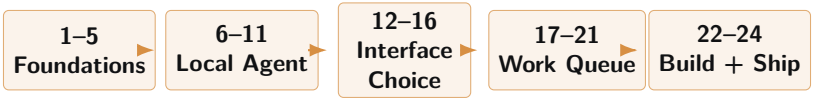
2026

Why This Course Is About Codex Desktop

- Codex is not just autocomplete. It is a local agent that can read files, edit them, run commands, and produce artifacts on your machine.
- The course uses the **Codex Desktop app** as the main surface: projects, threads, review UI, terminal, plugins, skills, automations, worktrees, and browser previews.
- You do not need a technical background. The sequence starts with useful practical wins and ends with a deployed web app.
- My working bias: Codex is where most beginners should start because the model, limits, and Desktop workflow are unusually strong together.

Context: daily Codex use, Director of AI/ML at Payslice, and 1,700+ AI MBA students learning AI workflows.

What You're Going To Learn



- First we make Codex legible: what it is, how the app works, and how to steer it.
- Then we add tools: plugins, computer use, skills, MCPs, and subagents.
- Finally we turn that into a real project: issues, stack, worktrees, and shipping.

Part 1 of 5: Codex Desktop Foundations

Start with the app you will actually use.

1. **What Codex Actually Is**
   Agentic coding on your real machine
2. **Pricing & Installing the Desktop App**
   Get the app installed and usable
3. **Desktop App Interface Tour**
   Threads, projects, plugins, settings, terminal
4. **Context Windows & Compaction**
   Why long conversations degrade and how to recover
5. **Voice: Wispr Flow + Dictation**
   Use voice to steer Codex faster

Part 2 of 5: Local Agent Basics

Make Codex useful without making it reckless.

6. **Permissions, Guardrails & Terminal Basics**
   Know what Codex can do and how to constrain it
7. **AGENTS.md**
   Project instructions that Codex reads automatically
8. **The Five Practical Primitives**
   CLI, skills, MCPs, plugins, subagents
9. **Plugins and the Plugin Directory**
   Extend Codex with bundled capabilities
10. **Computer Use**
    Let Codex inspect and operate browser surfaces
11. **Skills Deep Dive**
    Turn repeatable workflows into reusable instructions

Part 3 of 5: Choosing the Right Interface

Decide when to use skills, MCPs, apps, or agents.

12. **Skill vs MCP**
    API access, MCP access, and when to build on top
13. **Subagents**
    Parallel explanations, research, and implementation tracks
14. **When to Use What**
    The decision ladder across tools
15. **Git & GitHub**
    Commits, branches, and PRs without memorizing commands
16. **Decide What to Build**
    Creator Carousel Studio becomes the course project

Part 4 of 5: From Plan to Work Queue

Turn the app idea into work Codex can execute.

17. **Cloud Delegation**
    Understand it, but use it deliberately
18. **GitHub Issues**
    Make durable, reviewable units of work
19. **Automations**
    Recurring checks and background reminders
20. **The Stack**
    Next.js + Convex + Vercel
21. **Worktrees**
    Independent workspaces for independent features

Part 5 of 5: Build and Ship

Use the workflow to build the capstone app.

22. **Build Creator Carousel Studio**
    Live Codex build using issues, worktrees, review, and PRs
23. **Use the Companion Skill**
    App for visual judgment, skill for repeatable agent work
24. **Deploy, Smoke Test, Wrap**
    Ship the app, verify the live surface, and close the course

SECTION 1

What Codex Actually Is

ChatGPT / Claude vs Codex

ChatGPT / Claude Web

Runs in a sandbox

SEALED SANDBOX

Run code snippets

Generate text

× Can't see your files

× Can't run commands

VS

Codex

Runs on your machine

YOUR ACTUAL COMPUTER

Read & edit your files

Run terminal commands

✓ Access your projects

✓ Install packages

What Codex can actually do

Example: turning 10 PDFs into a synthesis brief

- **Input:** 10 academic papers on AI and labor economics, sitting in a project folder.
- **Prompt:** "Read every PDF. Write a synthesis brief — themes where they agree, where they disagree, and the 3 strongest empirical claims with citations. Then convert to PDF with pandoc + eisvogel."
- **Output:** A clean, cited PDF. No code written by the user.
- **Time:** 4 minutes.

The model read the papers, wrote the markdown, executed `pandoc` as a real shell command on the author's machine, and saved the result to disk. That's the pattern.

Codex Is Not Only For Code

Information work

- Read and synthesize PDFs
- Draft email replies from context
- Turn transcripts into briefs
- Prepare research packets

Operational work

- Clean folders and rename files
- Update spreadsheets
- Parse tax or finance documents
- Build small local tools

Mental model: advice chatbot $\rightarrow$ local collaborator with hands.

SECTION 2

Pricing & Installing

ChatGPT plans that include Codex

- **Plus — $20/mo.** Runs fine for short sessions, but the usage cap bites fast once you're working seriously.
- **Pro — $100/mo.** The sweet spot for most viewers. 5× Plus throughput, plus ChatGPT Pro bundled (GPT-5.4 Pro, unlimited Instant + Thinking).
- **Pro — $200/mo.** For full-time use. The course author is on this plan for daily work.

Recommendation: start at $100. Plus is too tight once you're using Codex seriously; $200 is overkill until you're coding every day.

Sign in with ChatGPT, not an API key — cloud threads (covered in §17) require the ChatGPT login.

How To Choose A Plan

- **Minimum: Plus.** Good for trying Codex and short sessions.
- **Default recommendation: Pro $100.** Enough headroom for real work without immediately running into weekly or five-hour-window limits.
- **Heavy daily use: Pro $200.** Useful when you run long sessions, subagents, review loops, or parallel worktrees every day.
- **Check your actual limits:** Settings → rate limits / usage remaining. The app UI matters more than memorizing a table.

For this course, subscription usage is the intended path. API-key billing is a separate product path, not the setup I recommend for following along.

Install checklist

1. **Download Codex Desktop.**
   `developers.openai.com/codex/app` — DMG on macOS, native installer on Windows. (Linux: waitlist as of recording.)
2. **Sign in with ChatGPT** on first run — not with an API key.
3. **Install VS Code** separately (`code.visualstudio.com`).
   Just as a file viewer / plain editor. No extensions needed. Any editor works — Cursor, Sublime, etc.
4. **Make a sandbox folder:** `~/Documents/codex-sandbox`, then open it as a project in Codex Desktop.

Companion tools (git, pandoc, node) are installed as they come up later in the course — every thread has its own terminal.

Optional Public Exercise: First Prompt

> *"Write me a single-file personal website with my name and a short bio. Save it as `index.html`. Open it in my browser when you're done."*

You should see:

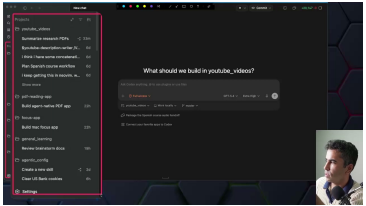
- A short tool-call stream (Codex thinking, writing the HTML, saving, opening).
- A personal website in your browser in under 2 minutes.
- No code written by you.

This is the pattern for the whole course: describe what you want; Codex does the work.

SECTION 3

Desktop App Interface Tour

The Codex Desktop interface at a glance



- **Left rail:** projects (folders on your machine) and the thread list for the current project.
- **Composer (bottom):** where you type, paste, dictate, drag files. Model and approval controls live in the bottom row.
- **Review pane (right):** diffs, staging, commits — the git UI inside Codex.
- **Pop-out threads:** any thread can open into its own window. Previews the parallel-agents workflow in §15.

Composer shortcuts & controls

- `/` — slash menu: built-in commands + skills.
- `@` — at menu: plugins (§9), subagents (§13), files, and references.
- `$` — skills only. Shortcut for the /-menu skills subset.

Bottom-row controls (per thread):

- **Model picker** — swap models mid-thread (e.g. GPT-5.3-codex).
- **Reasoning effort** — low / medium / high. Affects time and cost.
- **Approval toggle** — unified sandbox + approval. Default is Codex's recommended default.
- **Token counter** — fills as you use the context window; covered in §4.

Where The Non-Obvious Controls Live

- **Search:** left rail. Use it to recover old threads and decisions.
- **Plugins, Skills, Automations:** left rail. These are product surfaces, not hidden config files.
- **Terminal:** `Cmd+J` / `Ctrl+J`, scoped to the active thread.
- **Git review UI:** appears once the folder is a Git repo. Use it to inspect diffs, stage files, commit, and later push or open PRs.
- **Archive / updates / settings:** project and app settings menus.

The rest of the course keeps returning to these same surfaces. Learn the map once; reuse it everywhere.

Config directories

- `~/.codex/` — Codex-specific config: `config.toml`, `agents/`, `hooks/`, subagents.
- `~/.agents/` — portable skills (the cross-tool standard shared with other agent tools).

Repo-level overrides: `.codex/` and `.agents/` inside a project override user-level for that project. Commit them to share with teammates.

Heads-up: `~/.codex/skills/` does NOT work — skills only live under `.agents/skills/`. Put one in the wrong folder and it won't show up.

Closing exercise — chain them together

1. Drag a PDF into the composer; watch it preview inline.
2. Open the / menu; switch model to something different mid-thread.
3. Send this prompt: "Read this PDF. Update `index.html` to list the top five skills or themes from it as a new section."
4. Watch the token counter in the corner fill as the PDF is ingested.
5. Open `index.html` in your browser — the new section is there.

Five actions: compose, attach, swap models, write to a real file, watch context fill. That's ~80% of what you'll do in every session.

SECTION 4

Context Windows & Compaction

What is a context window?

- Measured in **tokens** — roughly related to words.
- **Large local models:** roughly 250k+ tokens, depending on model and app version.
- **Experimental flag:** up to 1M tokens. Leave it off unless you have a specific reason — longer context means more quality degradation.
- Every long session fills this up. You feel it: Codex starts forgetting files, decisions, project context.

What uses your tokens

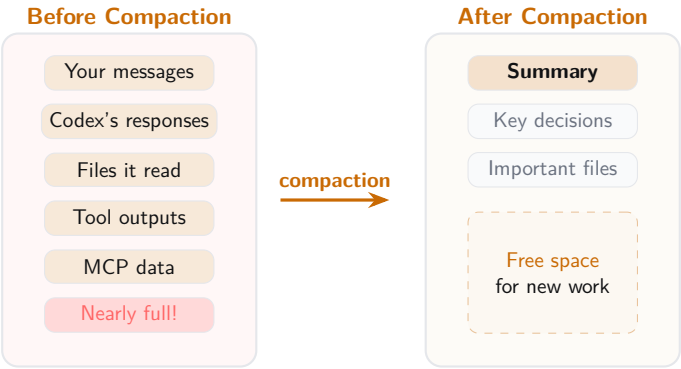
- **Your messages** — everything you type.
- **Codex's responses** — everything it types back.
- **Files it reads** — bigger file, more tokens.
- **Tool outputs** — shell commands, code searches, web fetches.
- **External data** — MCPs, skills, plugins. The fetched data usually costs much more than the call itself.
- **System prompt overhead** — your `AGENTS.md`, skill descriptions, MCP definitions. Loads on every request.
- **Thinking tokens** — included on subscriptions; stripped between turns so they don't pile up.

Models degrade at long context

The rule isn't "pick the best long-context model."
The rule is "don't let your threads get that long."

Reference: Anthropic's MRCR v2 benchmark — the pattern applies across providers. See `claude.com/blog/1m-context-ga` for the chart.

Compaction


Before Compaction
Your messages
Codex's responses
Files it read
Tool outputs
MCP data
Nearly full!
compaction
After Compaction
Summary
Key decisions
Important files
Free space
for new work

Compaction: Codex's escape hatch

- Codex **summarizes** the conversation so far.
- Keeps the summary, **drops the raw history**, frees up room.
- Same thread continues. Takes a minute or two.
- Compaction **loses some nuance** — that's the trade-off.

Trigger it yourself: type `/compact` in the composer. No button, composer-only.

When To Compact Manually

- Use manual `/compact` when you want to steer what survives.
- Good manual instruction: "compact this thread, but preserve the plan, file paths, open issues, and the decisions about PostHog."
- Start a fresh thread when the old one contains too many abandoned branches, stale assumptions, or unrelated investigations.

Rule: compaction saves momentum; it does not preserve every nuance.

What to do about it

- **Model choice** — cheaper/faster models for grunt work; save the big one for hard turns.
- **Reasoning effort** — low / medium / high. Lower burns less context and less time.
- **Compaction** — let auto-compact handle normal long sessions; use `/compact` when you want to steer the summary.

That's enough to manage context in practice. The rest of the course leans on this: every long skill, plugin, or subagent session has the same shape.

SECTION 5

Voice Mode & Wispr Flow

Voice: the fastest way to give Codex context

- **Typing:** ~40 words per minute.
- **Speaking:** ~150 words per minute.
- That's **4x more context per minute** just by talking.
- When you speak you naturally explain *why*, give background, mention edge cases. When you type you don't.

Recommendation: make voice your default way of working with Codex.

Typed vs spoken — same intent, different result

Typed

"fix the login bug"

Spoken

"There's a bug where users who sign in with Google get redirected back to the login page instead of the dashboard. It started after we added the OAuth callback — check the redirect logic and the session cookie, and if we're dropping the state param, fix that too."

Same person, same intent. Spoken version gives Codex enough to solve it first try. Typed version gets a generic guess.

Two ways to dictate into Codex

- **Wispr Flow** — system-wide dictation layer for Mac. Paid. Works in every text field on your computer: Codex, browser, Slack, terminal. Recommended if you're going to lean on voice daily.
- **Ctrl+M (built-in)** — with the Codex Desktop composer focused, press `Ctrl+M` to start; press `Ctrl+M` again to stop and transcribe. Toggle, not hold. Free, already installed, composer-only.

Either one works. Codex processes voice-dictated prompts identically to typed ones — there's no separate "voice mode" in the app; it's just text input.

What just happened

- **The prompt was ~60 words.** Fifteen seconds to speak; ~90 seconds to type. Most people would type "organize my screenshots" and get a worse result.
- **The richer prompt carried intent.** Categories, uncertainty handling, and desired summary were all part of the request.
- **The file-system reach belongs in the next section.** When Codex touches files outside the project, permissions and approval mode determine what happens.

Which brings us to permissions — the next section.

SECTION 6

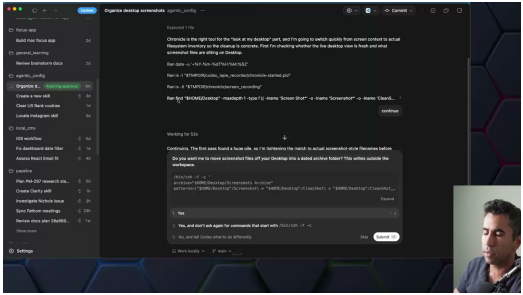
Permissions, Guardrails & Terminal Basics

Codex Desktop: two approval modes

- **Default Permissions.** Codex can read/edit files inside the current project and run routine commands there. Anything outside the workspace, or with side effects beyond it, **pauses and asks**.
- **Full Access.** Codex can touch any file, run any command — no popups. This is the mode the §5 voice demo used to reach into `~/Desktop`.

Rule of thumb: live in Default day-to-day. Flip to Full Access when you know exactly what you're asking for and don't want to be interrupted.

Approval Prompt In Practice



- Approve once for the specific command.
- Use “don’t ask again” only when the command pattern is clearly safe.
- Full Access removes these pauses, so add guardrails before using it for long runs.

Codex CLI permission matrix (for reference)

Sandbox tiers (`-sandbox`):
`read-only` | `workspace-write` (default) | `danger-full-access`

Approval policies (`-ask-for-approval`):
`on-request` (default) | `never` | `untrusted`

Starlark rules engine (`~/.codex/rules/default.rules`):
write allow / prompt / forbidden rules for specific command shapes. Experimental. Not covered in this course.

Docs: `developers.openai.com/codex/agent-approvals-security` & `developers.openai.com/codex/rules`

Terminal inside Codex Desktop

- **Per-thread terminal:** `Cmd+J` (macOS) / `Ctrl+J` (Windows) with a thread focused.
- Opens a real shell rooted at the current project folder.
- Same shell you'd get from `Terminal.app` or `Windows Terminal` — just docked inside the app.

For this course: think of the terminal as an **installer**, not a Codex interface. The composer is where Codex work happens.

Experimental features & `hooks`

The one we need for dcg: `hooks`

- Enables lifecycle hooks from `~/.codex/hooks.json` (or project-level `.codex/hooks.json`).
- Supports `SessionStart`, `PreToolUse`, `PostToolUse`, `UserPromptSubmit`, `Stop`.
- **Windows:** hooks currently disabled per docs.

Enable (no UI toggle — config.toml only):

1. Settings (gear) → Configuration tab → **Open config.toml**.
2. Add `hooks = true` under `[features]`, save.

Permission Layers In Practice

Desktop approval mode: Default Permissions or Full Access

Advanced config: network, read/write deny lists, rules

Hook guardrail: dcg rejects destructive commands before execution

Heavier options: Docker, isolated machines, separate accounts

My practical default: start in Default. When the interruptions become the bottleneck, use Full Access plus Destructive Command Guard.

Destructive Command Guard (dcg)

What it catches (default pack):

- `rm -rf` outside temp directories
- `git push -force`, `git reset -hard`, `git clean -f`, `git stash drop`
- `DROP TABLE`, `TRUNCATE` (Postgres pack)

Why it matters beyond the approval toggle: in Full Access mode the toggle stops asking. dcg sits **underneath** the toggle — it fires on destructive commands regardless of approval state.

Caveat per repo: a motivated agent could write the destructive command into a shell script to bypass the pattern match. Catches typos, confused autopilots, accidental paths — not adversarial behavior.

Repo: `github.com/Dicklesworthstone/destructive_command_guard`

Takeaways

1. **Default Permissions** for daily work. **Full Access** for "I know what I'm doing" moments. Toggle in the composer bottom row.
2. **You don't need a separate terminal** for install tasks — `Cmd+J` / `Ctrl+J` opens a real shell inside Codex Desktop.
3. **Install Destructive Command Guard.** One minute. Catches the destructive commands the approval toggle won't when you're in Full Access.

Everything else in permissions (CLI matrix, Starlark rules, writing your own hooks) is optional depth — links on the next slide and in the description.

Go deeper (optional)

- **CLI permission matrix** — sandbox tiers × approval policies.
  developers.openai.com/codex/agent-approvals-security
- **Starlark rules engine** — allow / prompt / forbidden rules for specific command shapes (experimental).
  developers.openai.com/codex/rules
- **Destructive Command Guard (repo)** — install, security packs, config.
  github.com/Dicklesworthstone/destructive_command_guard
- **Codex hooks (docs)** — lifecycle events, hooks.json schema.
  developers.openai.com/codex/hooks

SECTION 7

AGENTS.md

Where Codex looks for AGENTS.md

`some/path/AGENTS.md`	Walked down from project root
`./AGENTS.md`	Your team, via git
`~/.codex/AGENTS.md`	Just you, all projects

more specific

All levels concatenate — deeper files override earlier guidance

`AGENTS.override.md` replaces `AGENTS.md` at either scope if both exist

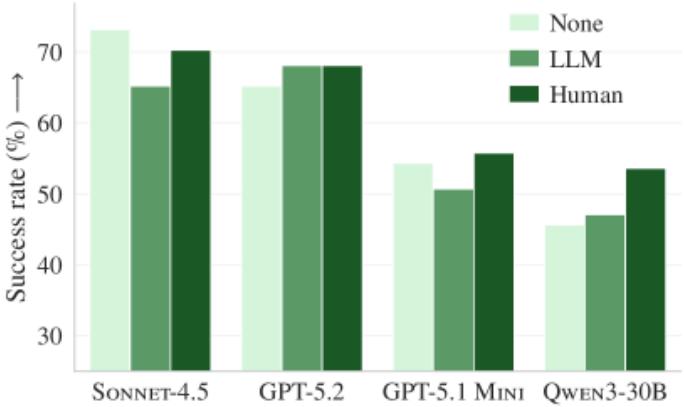
What Codex loads, in what order

- **Global:** `~/.codex/AGENTS.md` — just you, every project.
- **Project root:** `./AGENTS.md` — shared with your team via git.
- **Walked-down:** any `AGENTS.md` on the path from project root to your current directory. Deeper files override earlier guidance.
- **Override variant:** `AGENTS.override.md` replaces `AGENTS.md` at either scope if both exist. Useful for local-only overrides.

Size cap: 32 KiB by default (`project_doc_max_bytes`). Raise in config or split across nested dirs if you hit it.

Source: `developers.openai.com/codex/guides/agents-md`

Do AGENTS.md files actually help?



Source: Gloaguen et al. (2026), "Evaluating AGENTS.md," ETH Zurich / INSAIT

The cost of context files

		Sonnet-4.5		GPT-5.2		GPT-5.1 M.		Qwen3-30B	
		Steps	Cost	Steps	Cost	Steps	Cost	Steps	Cost
SWE-Bench Lite	None	54.4	1.30	12.5	0.32	40.9	0.18	29.7	0.12
	LLM	57.2	1.51	12.7	0.43	45.2	0.22	32.2	0.13
Agent-bench	None	40.7	1.15	12.1	0.38	40.6	0.18	31.5	0.13
	LLM	46.5	1.33	13.1	0.57	46.9	0.20	34.2	0.15
	Hum.	45.3	1.30	13.6	0.54	46.6	0.19	32.8	0.15

LLM-generated files increase steps and cost across every model.

Why LLM-generated AGENTS.md hurts

1. **It restates what Codex already discovers.** "This is a Next.js project with TypeScript" is in `package.json`. Writing it in AGENTS.md adds tokens without adding information.
2. **It goes stale.** Your project structure changes; your AGENTS.md doesn't. Theo from t3.gg tested this live — his old AGENTS.md told Codex to place files in directories that no longer existed. The file was *actively making things worse*.

Theo demo: youtube.com/watch?v=GcNu6wrLTJc

Do Not Start With AGENTS.md

- **Good:** stable constraints. "Use Bun, not npm." "Run tests before committing." "Tailwind v4, not v3."
- **Bad:** descriptions of current state. Architecture summaries, table counts, file structure — they go stale and mislead.

Length: under 200 lines is a good ceiling. Codex hard-caps at 32 KiB.

AGENTS.md vs skills

- **AGENTS.md = constraints.** "Use Bun, not npm." Loads every session, every turn. Adds to your base context.
- **Skill = workflow.** "How to prep for a meeting." Loads only when you invoke it. Doesn't bloat context the rest of the time.

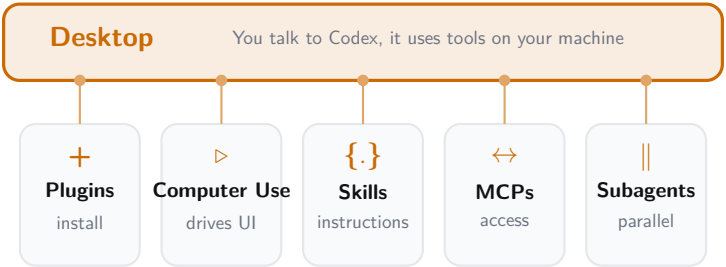
Rule of thumb: if you catch yourself writing a *procedure* in AGENTS.md — a multi-step checklist or "first do X, then Y" — stop. That's a skill.

Skills: covered in depth in the next sections.

SECTION 8

The Five Practical Primitives

The Capability Toolbelt



The Five Primitives at a Glance

- **Plugins** — bundles of capability others built. Install, go.
- **Computer Use** — first-class plugin. Codex drives your mouse and browser.
- **Skills** — your own reusable workflows, packaged up.
- **MCPs** — connect Codex to outside systems (Gmail, Calendar, Notion, APIs).
- **Subagents** — run multiple Codex instances in parallel.

The next five sections go deep on each, starting with Plugins.

Which Primitive Should I Reach For?

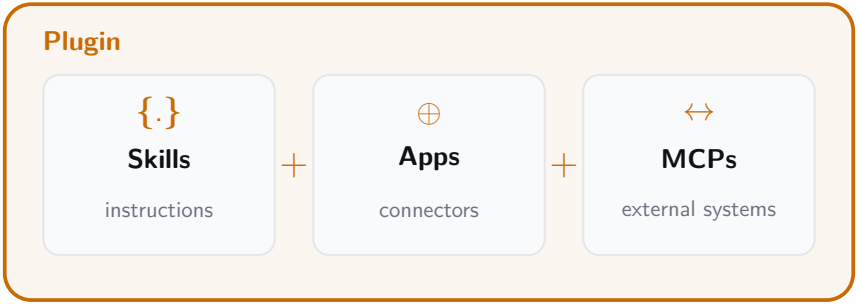
What is missing?	Reach first for
Immediate connection to an existing app	Plugin
Logged-in browser or GUI-only workflow	Computer Use
Personal SOP or repeatable procedure	Skill
Durable external-system access	API or MCP
Independent parallel investigation/work	Subagent

Order for most beginners: install a few useful plugins, then learn skills deeply.

SECTION 9

Plugins

What's Inside a Plugin



Not bundled: Subagents (see §13). Hooks are separate too.

What Plugins Are

A plugin is a bundle. Three things can be inside:

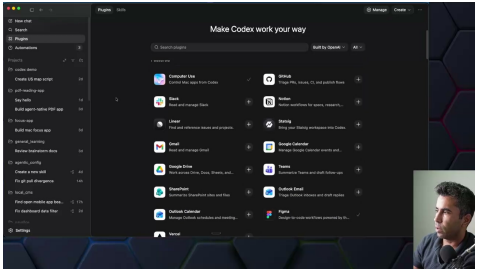
- **Skills** — reusable instructions for specific kinds of work
- **Apps** — connectors to Gmail, Google Drive, Slack, GitHub, etc.
- **MCPs** — services that give Codex access to external tools

Not bundled: Subagents are their own primitive (covered later). Hooks are separate too.

Marketplaces: plugins come from marketplaces. Default is OpenAI-curated. Third-party marketplaces are supported — `gmail@openai-curated` shows the marketplace after the `@`.

Starter picks from the curated directory: Gmail, Google Drive (Drive + Docs + Sheets + Slides), Slack, Notion.

Plugin Detail Pages Show The Bundle



- Inspect what the plugin is composed of: app connector, MCP server, skills, or a combination.
- This is why many “good MCP use cases” show up to the user as plugins.
- Install the package when it exists; build your own skill when your workflow needs more specific behavior.

Install And Use Plugins

Three install paths (pick whichever you prefer):

- **Desktop, left rail:** open Plugins menu → browse → click install
- **Desktop, composer:** type `/plugins` → same directory inline
- **CLI:** `codex /plugins` → terminal browser with marketplace tabs

Some plugins auth at install. Others on first use.

Using a plugin in a prompt:

- **Natural language** — just describe the task; Codex picks the right plugin
- `@plugin-name` — pin a specific plugin or bundled skill to a prompt

Control And Disconnect Plugins

Approvals still apply. Installing a plugin does not override your permission settings. Default mode still prompts before writes.

Disable without uninstall: edit `~/.codex/config.toml`:

```
[plugins."gmail@openai-curated"]
enabled = false
```

Restart Codex. Plugin stays installed, just turned off.

Uninstall caveat: removing a plugin from Codex does *not* disconnect the bundled app (Gmail, Notion, etc.) from your ChatGPT account. Manage connected apps in ChatGPT.

Build Your Own & Going Further

You can build and distribute your own plugin.
Scaffolding, manifests, and marketplace publishing are all documented:

`developers.openai.com/codex/plugins/build`

A community-built plugin worth knowing:
Compound Engineering — a framework for AI-assisted app building (ideation → brainstorm → plan → work). We install it and use it later in this course to build the full app.

Forward reference: this plugin gets its own section later in Act III, before the app build.

Coming up next:
The most first-class plugin of all has its own settings tab and its own section — **Computer Use**, where Codex drives your mouse and browser.

Plugins Are Useful, But Generic

- They usually handle the common access/auth/tool surface well.
- They may struggle with your exact project conventions, output shape, pagination edge cases, or business rules.
- That limitation is the bridge to skills: a skill can tell Codex how *you* want to use the plugin/API/MCP.

Practical pattern: plugin for access, skill for procedure.

SECTION 10

Computer Use

Computer Use: Setup

Platform: macOS only at launch. Not available in EEA, UK, or Switzerland.

Enable:

- Settings → Computer Use → Install
- Grant macOS **Screen Recording** + **Accessibility** permissions
- Codex asks per-app the first time, with an Always Allow option

Use it:

- Natural language, or mention `@Computer Use` or `@AppName`

File edits and shell commands still follow your sandbox and approval settings.

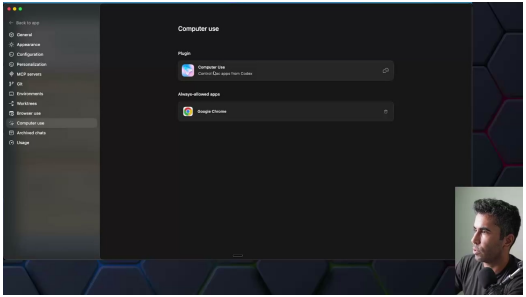
Computer Use: When To Reach For It

Decision tree:

- Plugin exists → use the plugin
- MCP exists → use the MCP
- Previewing your own local web app → in-app browser (`Cmd+Shift+B`)
- Everything else (apps without plugins, logged-in sites, multi-app workflows, GUI bugs, session-replay tools like Microsoft Clarity) → Computer Use

Hiccups to expect: misclicks on dense UIs, cookie banners derailing flows, slow pages compounding waits, and high token cost from per-step screenshots. Supervise on non-trivial tasks.

Computer Use When APIs Are Painful



- X/Twitter: browser works; API access can be awkward or expensive.
- Clarity/session replay: useful data often lives behind a logged-in UI.
- DaVinci Resolve or local apps: no clean MCP/plugin path.
- Local app testing: the screen is what you need Codex to inspect.

Decision rule: use clean tools when they exist; use Computer Use when the screen is the tool.

Computer Use: Safety & Going Deeper

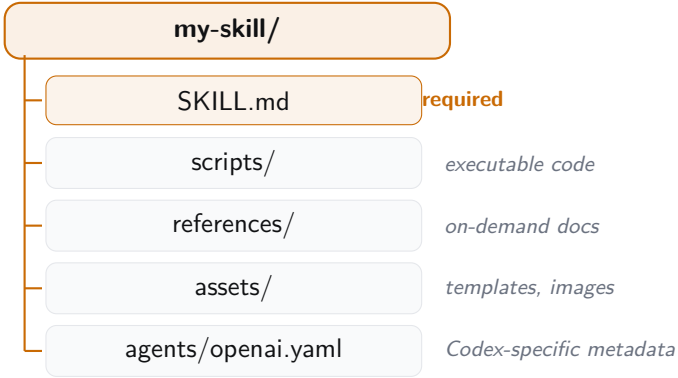
Safety notes:

- CU can see screen content, screenshots, clipboard, keyboard input — treat that as context Codex processes
- File edits and shell commands still follow your sandbox & approval settings
- Approvals per-app: first use asks, "Always allow" skips future prompts
- Revoke app access from Settings → Computer Use
- Cannot automate terminal apps, Codex itself, admin auth, or OS permission prompts
- Stay present for credentials, payments, account settings
- Can't stop mid-task? You can take over your computer at any point
- Browser (per docs): CU defaults to your main browser with its logged-in session. Review actions as if you took them yourself. Point it at a different browser to preserve your main session.

SECTION 11

Skills Deep Dive

Skill Anatomy



Only **SKILL.md** is required. Everything else is optional.

SKILL.md: Two Parts

SKILL.md

YAML frontmatter

name + description both required

```
--
name: my-skill
description: Use when the user asks for ...
--
```

Markdown body

Instructions Codex follows

```
# Steps
1. First do this ...
2. Then do this ...
```

Progressive Disclosure

1. Metadata – `name` + `description` — ~100 tokens/skill

Loaded at startup for every skill Codex knows about

2. SKILL.md body — <5000 tokens (recommended)

Loaded only when Codex decides to use the skill

3. Files in `scripts/` / `references/` / `assets/` — as needed

Loaded only when the skill's instructions explicitly reference them

Three Skill Scopes

Scope	Path	Use for
REPO	`.agents/skills/` walked up from CWD to repo root	Project-specific skills. Team commits to the repo.
USER	`~/.agents/skills/` *Docs say this. Installer writes to ~/.codex/skills/ — drift.*	Personal skills that work across all projects.
SYSTEM	`bundled with Codex` `skill-creator`, `plan`, `skill-installer`, ...	Always available. You don't manage these.

agents/openai.yaml — Block 1: interface

UI polish in the Codex app.
Display name, short description, icons, default prompt. Without yaml, the skill shows its folder name only.

Real example — OpenAI's Figma skill:

```
interface:
  display_name: "Figma"
  short_description: "Use Figma MCP for design-to-code work"
  icon_small: "./assets/figma-small.svg"
  icon_large: "./assets/figma.png"
  default_prompt: "Use $figma to inspect the target design..."
```

agents/openai.yaml — Block 2: policy

Invocation policy.
One field worth knowing: `allow_implicit_invocation`. Default `true`. Set `false` when you want to block auto-triggering.

```
policy:
  allow_implicit_invocation: false
```

Use when:

- Skill costs money per invocation
- Calls external APIs you pay for
- Does something you want to always explicitly confirm

`$skill-name` still fires manually. Just won't auto-trigger on description match.

agents/openai.yaml — Block 3: dependencies

Declared MCP dependencies.
Skill declares what MCP servers it needs to work. Codex uses this for a smoother setup experience.

Real example (Figma skill again):

```
dependencies:
  tools:
    - type: "mcp"
        value: "figma"
        url: "https://mcp.figma.com/mcp"
        transport: "streamable_http"
```

Deeper on MCPs in §12.

Skills: What They Are

A skill is a folder:

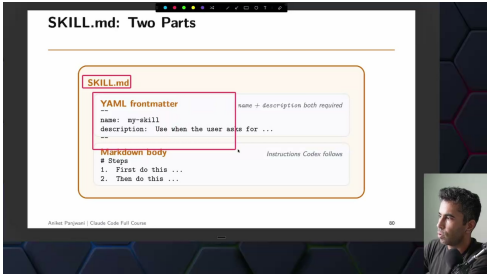
- `SKILL.md` — required. YAML frontmatter (`name`, `description`) + markdown instructions.
- `scripts/` (optional) — executable code the skill runs
- `references/` (optional) — on-demand docs
- `assets/` (optional) — templates, images
- `agents/openai.yaml` (optional) — Codex-specific metadata

Progressive disclosure (three load tiers):

- Metadata ($\sim$100 tokens/skill) always loaded
- SKILL.md body loaded only when skill is used
- Files in subdirectories loaded only when referenced
- Recommendation: keep SKILL.md body under 5000 tokens

Write a good `description`: Codex uses it to decide when to implicitly trigger the skill.

A Skill Is A Folder Codex Can Inspect



The description matters most: it is the always-loaded trigger text Codex uses to decide whether the skill should appear or fire implicitly.

Skills: Scopes, Invocation, Control

Three scopes:

- **Repo:** `.agents/skills/` walked from CWD up to repo root
- **User:** docs say `~/.agents/skills/`; installer currently writes to `~/.codex/skills/` — path drift, both worth knowing
- **System:** bundled with Codex. `skill-creator`, `plan`, `skill-installer`, etc.

Three ways to invoke: `$skill-name` explicit / `/skills` menu / implicit via `description` match.

Build + install: `$skill-creator` (guided builder) / `$skill-installer` (curated or repo-hosted skills). Both system-bundled.

Common pattern — wrap a CLI tool: Codex has shell access, so anything in `$PATH` (`yt-dlp`, `ffmpeg`, `gh`, `jq`) becomes a capability. A skill packages the invocation into a reusable workflow.

Enable/disable: flip the toggle in the Desktop app's **Skills** menu (left rail).

You Probably Do Not Need To Buy Skills

- Use `$skill-creator` for personal workflows before searching for a paid generic skill.
- Search GitHub or the web when you want examples, not authority.
- Keep the first version small: one trigger, one workflow, one output shape.
- Disable skills when they conflict, add context noise, or fire too broadly.

Useful skill demo pattern: `yt-dlp` gets transcript/metadata, the skill writes an image brief, `$imagegen` makes the visual.

Skill Installer Gotcha

- Repo skills should live under `.agents/skills/`.
- User-level skill locations and installer behavior have changed over time.
- After installing, restart Codex if the skill list does not refresh.
- If it still does not appear, inspect the installed folder and move/copy it into the location the current app recognizes.

This is boring, but it saves the exact five-minute confusion that makes skills feel broken.

`agents/openai.yaml`: Codex-Specific Metadata

1. `interface` — UI polish in $ / `/skills` menus:
`display_name`, `short_description`, `icon_small`, `icon_large`, `default_prompt`.

2. `policy.allow_implicit_invocation` — default `true`. Set `false` to block auto-triggering. Users still invoke explicitly with `$skill-name`.

3. `dependencies.tools` — declare MCP servers the skill needs. Each entry: `type`, `value`, `description`, `url`, `transport`.

Honest caveat: `agents/openai.yaml` is Codex-specific — the spec allows extra files, OpenAI chose this path. Not guaranteed to be adopted by other harnesses.

Skills vs plugins: both are distribution formats. Ship one focused workflow as a skill; ship a cohesive set or first-party integration as a plugin.

Docs: `developers.openai.com/codex/skills` Spec: `agentskills.io` Examples: `github.com/openai/skills`

SECTION 12

Skill vs MCP

API → MCP → Skill

API

- Underlying access
- REST, GraphQL, SDK, local command
- What the service exposes

MCP

- Model-facing wrapper
- Tools and schemas Codex can call
- Often sits on top of an API

Skill

- Workflow layer
- Your rules, defaults, approvals
- Can call API or MCP underneath

Rule: API is access. MCP packages access for models. Skill controls behavior.

MCP In One Sentence

- The service still has some underlying access layer: API, SDK, database, local command, or browser-authenticated server.
- MCP exposes model-readable tools and schemas on top of that access.
- Plugins often package MCPs so the user experience is just "install the plugin."
- Skills can sit above the API or MCP and teach Codex your procedure.

Shortcut: API is access. MCP packages access. Skill controls behavior.

When an MCP is actually worth it

- **Live design or browser state:** Figma, browser control, Chrome DevTools.
- **OAuth-heavy workspaces:** Notion, Sentry, Linear, GitHub, internal tools.
- **Distributed integrations:** a formal tool interface for multiple users, agents, or apps.
- **Product surfaces:** AI MBA Pro exposes member materials through a remote MCP.
- **Maintained access layer:** someone else owns the API details and auth flow.

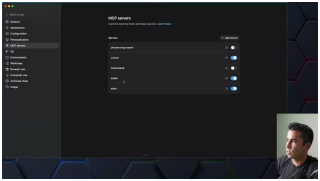
Still common: put a skill on top of the MCP so Codex follows your naming conventions, target databases, approval rules, and summary format.

Plugins Often Contain MCPs

- Open a plugin detail page and inspect what it is composed of.
- A plugin can bundle skills, app connectors, MCP servers, or all three.
- So the user action may be "install the plugin," while the access layer inside is MCP.

Rule: plugin is packaging; MCP is access; skill is workflow.

Configure MCPs in Codex Desktop

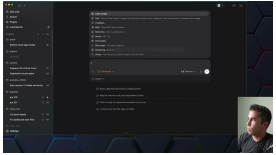


1. First inspect a plugin detail page to see its bundled capabilities.
2. Open **Settings** in Codex Desktop.
3. Go to the **MCP servers** area.
4. Add or inspect a custom remote server such as **AI MBA Pro**.
5. Add a hosted HTTP server by URL, or a local server by command.
6. Complete OAuth in the browser when the server requires it.

Demo posture: show the UI shape and where the connection lives.

MCP Setup Checklist

1. Settings → MCP Servers → Add MCP Server.
2. Choose the shape: **local STDIO command** or **hosted HTTP**.
3. Enter command or URL; add headers only when the service asks for them.
4. Complete browser OAuth if the server supports it.
5. Use `/mcp` to check status and see which servers are available.



Two MCP Shapes

Hosted / HTTP

- Server already lives at a URL
- Example: AI MBA Pro MCP
- Often uses OAuth in the browser

Local command

- Codex starts a process locally
- Example: Context7 via `npx`
- Useful for local or developer tools

Same underlying idea: expose tools Codex can call.

MCP Demo Result: AI MBA Pro

- Tools: list resources, search resources, get resources, chunk long transcripts.
- Prompt shape: "search Pro group calls for MCP auth/setup advice and cite sources."
- Proof of value: Codex can return timestamps, quotes, and source references.
- Next layer: a skill can wrap the MCP into a personal summary/checklist workflow.

Lesson: MCP gives access; the skill turns access into a habit.

The Practical Rule

1. **Need access?**
   First ask if the service gives you a direct API, SDK, CLI, or local command.
2. **No clean direct access?**
   Use the MCP as the model-facing tool interface.
3. **Need your workflow followed?**
   Use a skill, whether it calls the API directly or calls the MCP.

Default for personal automation: start with a skill. Add API calls or MCP when access is the bottleneck.

Plugin vs Skill vs MCP

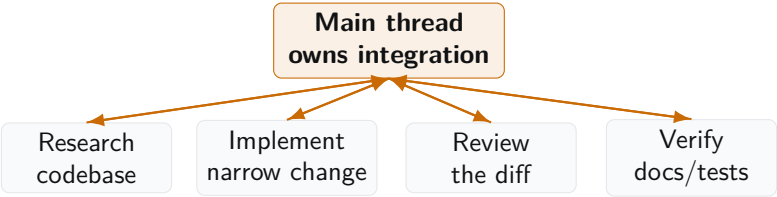
- **Plugin:** installable package. Can bundle skills, app connectors, and MCP servers.
- **API:** underlying service access.
- **MCP:** model-facing wrapper that exposes tools to Codex.
- **Skill:** workflow instructions. Teaches Codex how you want a job done.

Stack: API gives access; MCP packages it for models; skill controls behavior.

SECTION 13

Subagents

Subagents Are Parallel Lanes



Subagents are parallel attention, not magic coordination.

Fuzzy vs Divisible

Too fuzzy

- Figure out the strategy.
- Fix the app.
- Improve the workflow.

Divisible

- Inspect one module.
- Update one file family.
- Review one diff for one risk.

Boundary + output is what makes parallel work useful.

Built-In Agents

- **default:** general-purpose agent.
- **worker:** execution-focused; bounded code, docs, or test changes.
- **explorer:** read-heavy; codebase questions, patterns, and architecture.

Codex only spawns subagents when explicitly requested. Be deliberate.

Subagent Tradeoff

What you gain

- Separate context window
- Parallel research or implementation
- Smaller reports back to main thread

What you still pay

- Account/model usage
- Integration work in the main thread
- Confusion if the boundaries overlap

Good split first, parallelism second.

Be Explicit About Spawning

Explorer lane

Use an explorer to inspect the auth flow while the main thread stays focused on the UI change.

Worker lane

Spawn a worker for the docs update. It owns only `docs/pro-mcp-setup.md` and should not touch app code.

Ownership line: the difference between useful parallelism and overlap.

Delegate With Four Things

1. **Role** — worker, explorer, reviewer, researcher.
2. **Scope** — files, module, or question it owns.
3. **Output** — patch, findings, summary, checklist, changed files.
4. **Boundary** — what it should not touch.

Smell test: if two agents edit the same files, the split is probably wrong.

Delegation Prompt Pattern

```
Clone github.com/badlogic/pi-mono to a temp folder.
Spawn subagents by package.  Each lane returns:
- purpose
- key files / entry points
- relationships to other packages
- reading path for a beginner
- open questions
Main thread synthesizes one architecture map.
```

The important part: every agent has a package, an output shape, and a boundary.

Custom Agent Anatomy

```
name = "docs-reviewer"
description = "Reviews docs for accuracy and missing steps."
developer_instructions = """
You are a documentation reviewer.
Check commands, prerequisites, and likely reader confusion.
Return findings with file paths and concrete fixes.
"""
```

Required: `name`, `description`, `developer_instructions`.

The Orchestration Rule

Use subagents for

- Parallel codebase research
- Independent doc updates
- Review while implementation continues

Keep one thread for

- Early product thinking
- Ambiguous debugging
- Tightly coupled refactors
- One continuous line of reasoning

Skill vs Subagent

Skill

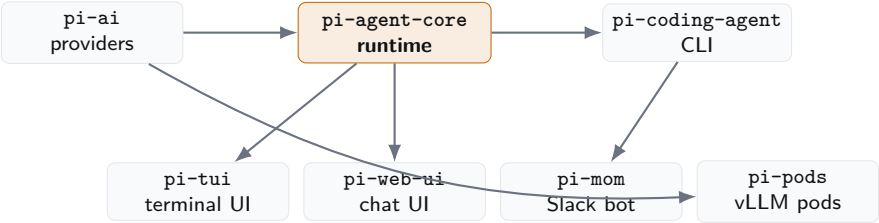
- Durable instructions
- Repeatable workflow
- Progressive disclosure
- Your preferences in one place

Subagent

- Parallel context
- Specialized role
- Bounded task
- Returns results to main thread

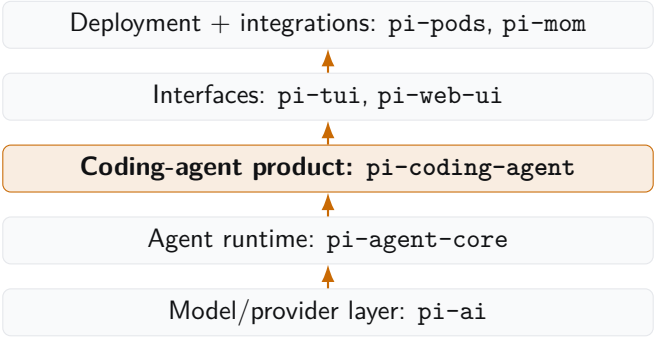
Pattern: skill defines the playbook; subagents run independent lanes.

Pi Package Lanes



Each lane returns: purpose, entry points, relationships, beginner note, open questions.

Pi Architecture Map



Subagents gather the pieces. The main thread names the system.

SECTION 14

When to Use What

The Missing-Piece Question

If what is missing is...

- Packaged capability
- Browser/desktop control
- Your workflow
- External access
- Parallel lanes

Reach first for...

- Plugin
- Computer Use
- Skill
- API or MCP
- Subagent

Not: which feature is coolest? **Yes:** what is missing?

Decision Chart: Primitives

Someone packaged the capability	**Plugin**
Needs real browser or app control	**Computer Use**
Codex needs your procedure	**Skill**
Needs reliable external access	**API / MCP**
Has independent side lanes	**Subagent**

Common Confusions

AGENTS.md vs Skill

AGENTS.md = constraints. Skill = repeatable procedure.

MCP vs Plugin

Plugin = installable bundle. MCP = one possible access layer inside it.

Skill vs MCP

Skill = how to do the job. MCP = access to a system.

Skill vs Plugin

Plugin = packaged capability. Skill = your workflow.

Simple rule: constraints, workflow, access, and packaging are different layers.

Plugin vs Computer Use

Plugin

- Clean packaged capability
- Known tool surface
- Reusable integration
- Prefer when available

Computer Use

- Logged-in browser/session
- Desktop apps
- Visual UI bugs
- Forms and messy surfaces

If clean access exists, use it. If the screen is the only API, use CU.

Autonomy Ladder

Automation: scheduled background workflow

Cloud task: remote async work, review later

Worktree threads: parallel full threads, isolated files

Subagents: parallel side tasks, main thread integrates

Skill: you stay in the loop, Codex has a playbook

Prompt: you ask, Codex answers

Power is not the goal. Fit is the goal.

One-Shot vs Recurring

Once	Repeated	Scheduled
Summarize this PDF.	Same sources, same output, same standards.	Run every Monday and send results to triage.
Prompt	**Skill**	**Automation**

If private systems are involved: add API or MCP access underneath.

Skill or App?

Skill is enough when

- One person uses it
- Text output is enough
- Local HTML/markdown review is enough
- Local state stays under your control

Build an app when

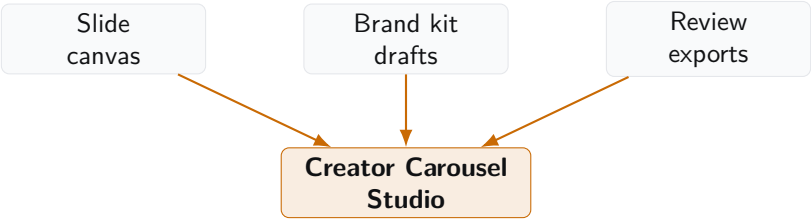
- Non-technical users need the workflow
- Team/public access matters
- Visual review is the main job
- Persistence becomes product state

Access Layer vs Workflow Layer

Access layer	Workflow layer
Direct API / SDK / CLI	Skill instructions around your defaults
MCP server	Skill on top for summary, naming, and checks
Plugin bundle	Prompt or skill that uses the plugin consistently
Logged-in screen	Computer Use with tight supervision

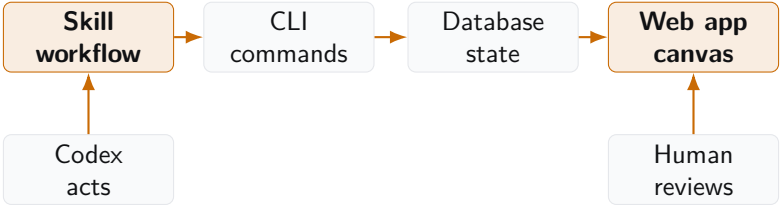
Most durable setup: pick the cleanest access layer, then encode your workflow as a skill.

Creator Carousel Studio Decision



A skill can draft copy. The app owns visual judgment and durable state.

App Plus Skill




```
create carousel from section 13 notes
check slide 4 for text overflow
```

Final Decision Rule

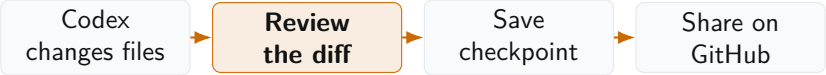
- Packaged capability → plugin
- Screen control → Computer Use
- Workflow → skill
- Access → API or MCP
- Parallel attention → subagents
- Persistence / visual product surface → app
- Scheduled execution → automation
- Isolated parallel dev → worktrees
- Remote async work → cloud task

Next: Git and GitHub, because every serious parallel workflow needs change tracking.

SECTION 15

Git & GitHub

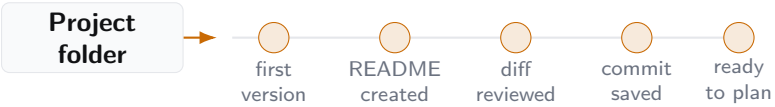
Why Git Matters



The goal is not command memorization. The goal is safe review.

What Git Is

Version control = save the history, not just the latest files

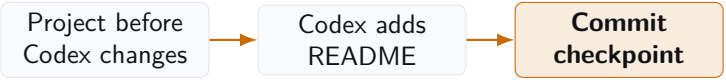


Minimum Git/GitHub Setup

- Install Git if needed: `brew install git`
- Install GitHub CLI if needed: `brew install gh`
- Authenticate once: `gh auth login`
- Ask Codex to verify: Git installed, name/email configured, `gh auth status` works
- Then ask Codex to initialize the repo, stage files, and create the first commit

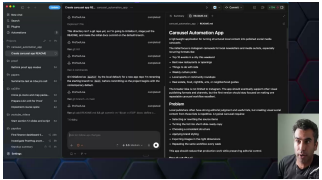
Practical stance: Codex runs the commands. You inspect the diff and approve the checkpoint.

Commit = Checkpoint



Commit after meaningful chunks: draft created, tests pass, UI looks right.

Desktop Diff Review



Ask Codex

- Show me the diff
- Stage this file
- Commit this checkpoint
- Explain why this changed

You check

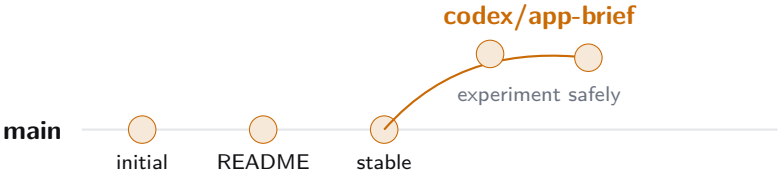
- Did only expected files change?
- Does the content match intent?
- Is anything unrelated included?
- Is this worth saving?

Desktop Review States

- **Unstaged:** files changed but not selected for the next commit.
- **Staged:** files selected for the checkpoint.
- **Last turn:** what Codex changed in the most recent assistant turn.
- **Commit / commit and push:** save locally, then send to GitHub when ready.

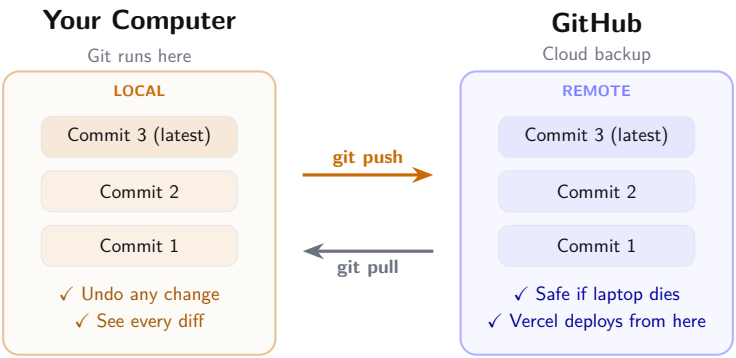
This is why Git helps beginners: you can review visually even when Codex runs the commands.

Branch = Experiment Line

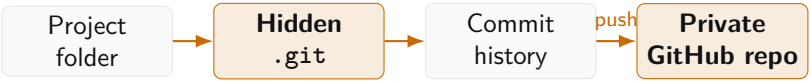


For now: know the concept. We will use branches practically after the work is planned.

Git vs GitHub

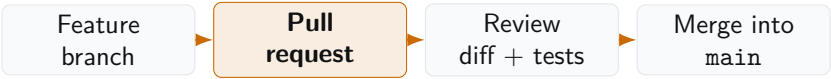


Local Repo To GitHub



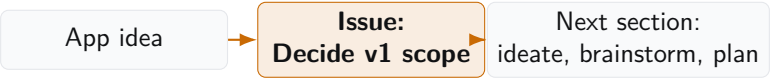
GitHub is where PRs, issues, reviews, and remote collaboration enter the workflow.

PR = Review Before Merge



For now: understand the review surface. Later sections will use PRs for real work.

Issues Are Named Work



Before worktrees: know what work exists.

The Practical Loop Today

Branches and PRs are vocabulary now. Practical use comes after planning.

Human vs Codex Responsibilities

Codex can operate

- Edit files
- Run tests
- Stage and commit
- Explain diffs
- Later: push branches and open PRs

You still review

- Does this belong?
- Are unrelated files included?
- Is the diff safe?
- Should this merge?

What to Remember

- Git is the safety layer for agentic coding.
- Commits are checkpoints.
- Branches are experiment lines.
- GitHub is the online home for those lines.
- PRs are review before merge.
- Issues are named units of work.
- Codex operates; you inspect the result.

Next: decide what the work is before using branches/worktrees practically.

SECTION 16

Decide What To Build

From App Idea to Work Items

Use Codex as a product-thinking partner before code generation.

The Four-Step Funnel

App idea

Direction

Product shape

Implementation plan

Work items

Idea → direction → shape → plan → issues.

Ideate vs Brainstorm vs Plan

`ce-ideate`	`ce-brainstorm`	`ce-plan`
What product directions are worth considering?	What should this chosen direction actually mean?	How should the work be built and verified?

This is product thinking before code generation.

Three Planning Prompts

1. **`ce-ideate`:** Explore product directions for Creator Carousel Studio. Compare demo value, build complexity, and how clearly each direction teaches Codex Desktop.
2. **`ce-brainstorm`:** Take the selected direction and flesh out v1 user flows, screens, states, non-goals, success criteria, and edge cases.
3. **`ce-plan`:** Turn the brainstorm into ordered milestones and implementation units with expected files, testing notes, and issue candidates.

Do not build yet. Create the map first.

Choose for Demo Value

Quick to understand

Meaningful but bounded build

Clarifies app vs skill

A skill can draft copy. An app can hold state, preview, assets, styles, and exports.

Planning Artifacts Become Context

Artifacts

- `docs/ideation/...`
- `docs/brainstorms/...`
- `docs/plans/...`

Why it matters

- Future threads can read them
- Decisions live beside the code
- The repo becomes shared memory

Why Write The Plan To Markdown?

- Humans can read and edit it before Codex builds.
- It survives compaction and can seed a fresh thread.
- It can be committed beside the code.
- It can become GitHub issues and later worktree prompts.
- It makes your product decisions inspectable, not hidden inside chat history.

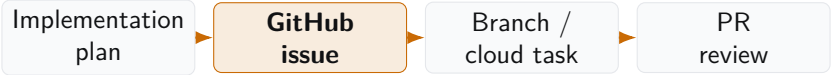
The repo becomes shared memory for you and future Codex threads.

Human Judgment Still Matters

- Ask clarifying questions when a plan sounds vague.
- Provide screenshots or reference examples when visual taste matters.
- Tighten non-goals before implementation starts.
- Rename the product when ideation reveals a better shape.

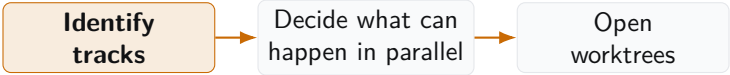
Creator Carousel Studio is the result of narrowing, not the starting prompt.

From Plan to Issues



Clear plan, clear issues. Bad plan, bad issues.

Why Not Worktrees Yet?



First decide the tracks. Then parallelize the tracks.

The Section 16 Output

- Ideation document
- Brainstorm document
- Implementation plan
- Candidate GitHub issues

This is what makes cloud delegation, issues, worktrees, and stack choices make sense.

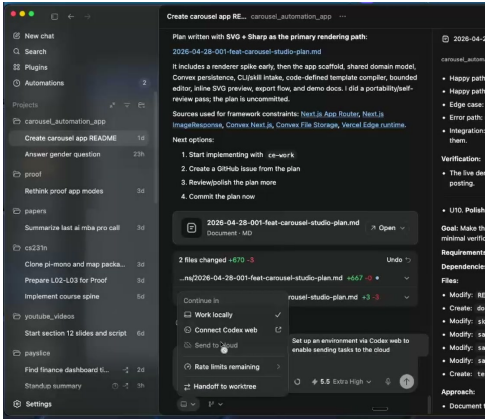
SECTION 17

Cloud Delegation

Cloud Delegation in One Sentence

Cloud moves execution elsewhere. It does not remove supervision.

Where You See It In Codex



- Open the `Continue in` menu
- `Work locally` stays in the current Desktop thread
- `Send to cloud` requires a configured cloud environment
- `Handoff to worktree` creates an isolated local checkout

Useful distinction: cloud changes where work runs; worktrees change which local checkout Codex edits.

What the Cloud Environment Actually Is

Cloud environments

Customize dependencies and tools for Codex

Copy Page

Use environments to control what Codex installs and runs during cloud tasks. For example, you can add dependencies, install tools like linters and formatters, and set environment variables.

Configure environments in Codex settings.

How Codex cloud tasks run

Here's what happens when you submit a task:

1. Codex creates a container and checks out your repo at the selected branch or commit SHA.
2. Codex runs your setup script, plus an optional maintenance script when a cached container is resumed.
3. Codex applies your internet access settings. Setup scripts run with internet access. Agent internet access is off by default, but you can enable limited or unrestricted access if needed. See agent internet access.
4. The agent runs terminal commands in a loop. It edits code, runs checks, and tries to validate its work. If your repo includes `AGENTS.md`, the agent uses it to find project-specific lint and test commands.
5. When the agent finishes, it shows its answer and a diff of any files it changed. You can open a PR or ask follow-up questions.

Source: OpenAI Codex cloud environments docs

- Container checks out the repo
- Setup script installs what it needs
- Internet access is configured
- Agent edits, runs checks, returns a diff

The hidden work: make the environment reproducible enough.

Why I Would Not Start Here

Local already has

- Your real files and tools
- Running dev servers
- Local auth and services
- Fast steering

Cloud needs

- Dependencies installed
- Setup/test commands
- Env vars and secrets
- Service access

Question: have I made a clean enough environment for the cloud agent to succeed?

Cloud Tasks Need Complete Prompts

- Define the repo, branch, files, bug, or issue clearly.
- State constraints and non-goals up front.
- Configure secrets, network, setup scripts, and tests before relying on it.
- Expect a returned diff, PR, or written analysis; review it like any other work.
- Do not assume it can pause for normal approval back-and-forth.

Local Codex is better when the task needs taste loops, live steering, or the strongest local model.

GitHub Review Is a Cloud-Backed Path

Use Codex in GitHub

Run Codex code review in pull requests

Copy Page

Use Codex to review pull requests without leaving GitHub. Add a pull request comment with `@codex review`, and Codex replies with a standard GitHub code review.

Source: OpenAI Codex GitHub integration docs

- Set up Codex cloud
- Enable code review for the repo
- Comment `@codex review`
- Or enable automatic reviews

Good when: the work already lives in a pull request.

Usage and Billing Reality

What are the usage limits for my plan?

The number of Codex messages you can send depends on the model used, size and complexity of your coding tasks and whether you run them locally or in the cloud. Small scripts or routine functions may consume only a fraction of your allowance, while larger codebases, long-running tasks, or extended sessions that require Codex to hold more context will use significantly more per message.

GPT-5.5 uses significantly fewer tokens to achieve results comparable to GPT-5.4. Its Codex setup runs faster and delivers higher-quality results for most users. These efficiency gains support generous usage limits despite GPT-5.5 being a significantly more capable model.

Plus | Pro 5x | Pro 20x | Business | API Key

	Local Messages* / 5h	Cloud Tasks* / 5h	Code Reviews / 5h
GPT-5.5	15-80	Not available	Not available
GPT-5.4	20-100	Not available	Not available
GPT-5.4-mini	60-350	Not available	Not available
GPT-5.3-Codex	30-150	10-60	20-50

The usage limits for local messages and cloud tasks share a **five-hour window. Additional weekly limits may apply.*

Source: OpenAI Codex pricing docs

- Cloud tasks use Codex usage from your ChatGPT plan or workspace credits
- GitHub PR review is also Codex product usage
- Normal API-key billing is a different path

Careful phrasing: no separate OpenAI API billing setup is required for Codex cloud tasks.

The Model Caveat

Local messages

- GPT-5.5
- GPT-5.4
- GPT-5.4-mini
- GPT-5.3-Codex

Cloud tasks + code review

- GPT-5.3-Codex
- GPT-5.5: not available
- GPT-5.4: not available
- GPT-5.4-mini: not available

Implication: if you need the strongest local model or lots of steering, keep the work local.

No Magic Infrastructure Layer

Codex settings

Setup scripts

Package versions

Env vars

Secrets

Internet access

Still your job to make the environment reproducible

The Practical Alternative: Local Review

Review

Review and iterate with Codex on changes inside the app

Copy Page

The review pane helps you understand what Codex changed, give targeted feedback, and decide what to keep.

It only works for projects that live inside a Git repository. If your project isn't a Git repository yet, the review pane will prompt you to create one.

What changes it shows

The review pane reflects the state of your Git repository, not just what Codex edited. That means it will show:

- Changes made by Codex
- Changes you made yourself
- Any other uncommitted changes in the repo

By default, the review pane focuses on **uncommitted changes**. You can also switch the scope to:

- **All branch changes** (diff against your base branch)
- **Last turn changes** (just the most recent assistant turn)

When working locally, you can also toggle between **Unstaged** and **Staged** changes.

Source: OpenAI Codex app review docs

- Review uncommitted changes
- Compare against a base branch
- See inline comments in the app
- Keep the whole fix loop local

For this course: `/review` is usually the simpler move.

Code Review Paths

GitHub review

- PR already exists
- Comment `@codex review`
- Or enable automatic reviews
- Uses code-review usage

Local review

- Working tree or branch diff
- Use `/review`
- Inline comments in the app
- Usually simpler for this course

The Dream Is High-Autonomy Work

- Cloud delegation is one product shape around this dream
- It is something to watch and think about carefully
- It is not my default recommendation for this course

Adjacent Directions

Cloud-agent products

- Devin and similar tools
- Cloud environment is the product surface
- Separate pricing / credits / setup assumptions
- Often ahead on delegation UX

Local / self-hosted orchestration

- OpenAI Symphony
- Isolated autonomous implementation runs
- Proof of work before acceptance
- Not the same as Codex cloud delegation

Shared direction: move from supervising coding agents to managing work.

What I Want You To Take Away

- The autonomy dream is real and worth tracking
- Codex cloud delegation is one current product path
- Today it comes with setup, model, usage, and review caveats
- For this course, the practical workflow is still local Codex Desktop

The work we do next still matters: clearer issues, reproducible projects, and better review loops make higher-autonomy delegation more realistic later.

SECTION 18

GitHub Issues

Why Issues Matter Now

Issues are a better target for Codex than vague repo scanning.

Issue Anatomy

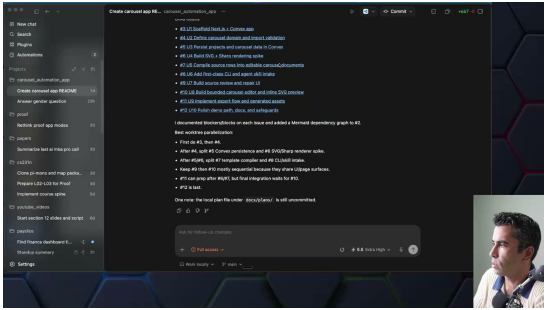
- What is the work?
- Why does it matter?
- What does done mean?
- What can make this fail?
- How should Codex verify it?

Title + context + acceptance criteria + blockers + verification.

Prompt: Turn A Plan Into Issues

```
Read docs/plans/creator-carousel-studio-v1.md.
Create GitHub issues for the first build milestone.
Each issue should include:
- source plan link / section
- context and user value
- acceptance criteria
- blockers or dependencies
- verification steps
- whether it is safe for a future worktree
Do not implement yet.
```

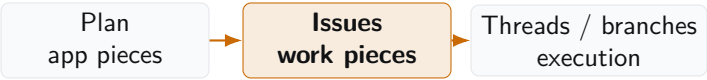
A Good Issue References The Source Plan



- Title names the concrete work
- Context links back to the plan
- Acceptance criteria define done
- Dependencies make blockers visible
- Verification tells Codex how to prove it worked

Why it matters: future threads can recover context after compaction or weeks away.

From Plan to Issues



- Ready?
- Blocked?
- Too vague?
- Independent enough for parallel work later?

SECTION 19

Automations

What Do Automations Mean in Codex?

Scheduled Codex task ► Runs in the background ► Findings go to inbox ► No findings? Archive

- You define the job once, then Codex runs it on a schedule.
- Project automations depend on the app and local project being available.
- Skills can carry the detailed workflow; the automation supplies the cadence.

Docs: Automations overview

Two Automation Shapes

Thread automation

Wakes this same conversation on a schedule.

Best when the existing thread context matters.

Standalone / project automation

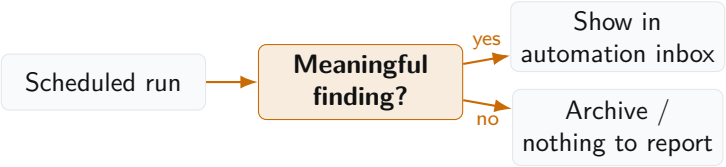
Starts fresh scheduled runs and reports results separately.

Best when each run should be independent.

If it “forgot” your thread, first ask which shape you created.

Docs: Thread automations and managing tasks

Where Results Go



Good prompts define what counts as a finding: blocked issues, stale PRs, failing checks, or a concrete next action.

Say this live: I would look for this in the Automations pane, not assume a visible sidebar item literally named Triage.

Docs: Managing tasks

Local Project or Worktree?

Local project

Convenient for read-heavy checks.

Risk: can touch files you are actively editing.

Background worktree

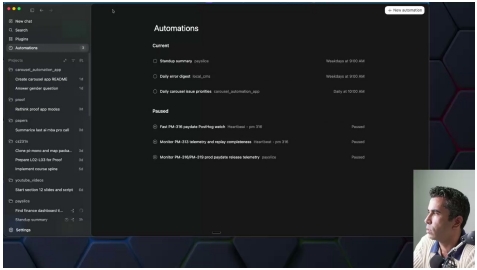
Isolates automation diffs from unfinished work.

Cleanup matters if schedules are frequent.

Rule: read-only steering can use local mode; if edits need isolation, choose a background worktree.

Docs: Local project vs worktree and worktree cleanup

Access Comes From Defaults



Key caveat: the automation page does not show a separate "permissions" picker; unattended runs inherit configured access and cannot rely on live approval back-and-forth.

Docs: Permissions and security model, app settings, and rules

Automation Prompt Pattern

```
Every weekday morning, in Creator Carousel Studio:
1. Review open GitHub issues.
2. Group into ready / blocked / unclear / parallelizable.
3. Recommend the next issue to work on.
4. Do not implement, close issues, or edit code.
5. Report only meaningful findings or one clear next action.
Use $creator-carousel-triage if available.
```

Pattern: cadence + project + tools + success criteria + output shape + guardrail.

Why Automations Fail

- Project or worktree is not available when the schedule runs.
- Required GitHub, plugin, MCP, or network access is missing.
- Credentials exist in your live thread but not in the automation environment.
- The job needs approval back-and-forth.
- Output shape is too vague, so Codex cannot decide what counts as a finding.

Fix: run manually once, tighten the job, and make access durable.

Why a Thread Task Can Fail Later

Live thread	Automation
Can ask follow-ups	Runs unattended
Can request approval	Uses configured sandbox
Has current conversation	Needs durable prompt + access

Diagnosis: not always a bad prompt; often a mismatch in context, project availability, tools, or permissions.

Docs: Thread automations and permissions

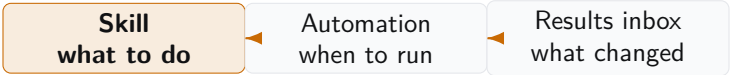
Test Before Scheduling

- Can Codex see the selected project?
- Can it use the required plugin or skill?
- Can it access GitHub or the needed network resource?
- Does the output match the shape you want?
- Can it finish without asking for extra permissions?

If not, narrow the job, change sandbox settings, use a worktree, or make it advisory.

Docs: Test automations

Skills Make Automations Maintainable



Example prompt: `Every weekday morning, run $creator-carousel-triage and report findings.`

Docs: Skills in automations and examples

A Useful Automation Job

- Review open GitHub issues.
- Group into ready, blocked, unclear, and candidate-parallel work.
- Recommend the next issue to work on.
- Do not implement or close issues.
- Report only meaningful findings or a clear next action.

This is a project-steering automation, not an unattended builder.

Docs: Automation examples

Automation vs Cloud vs Worktree

Cloud	Automation	Worktree
Send a clear task away once.	Run a recurring task on a schedule.	Isolate a workspace for a branch.

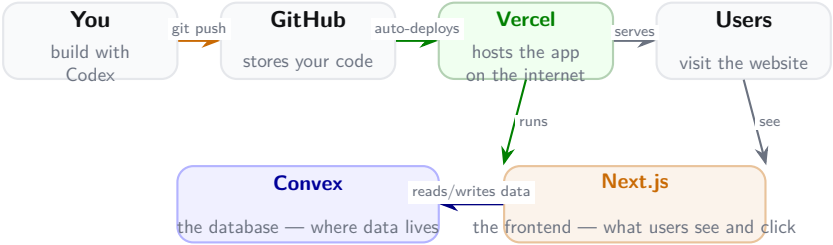
Issues identify the work. Automations keep checking it. Skills define the workflow. Worktrees isolate parallel execution.

Docs: Managing tasks and thread automations

SECTION 20

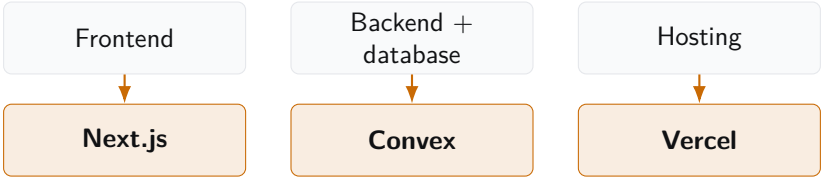
The Stack: Next.js + Convex + Vercel

Every Web App Has Three Layers



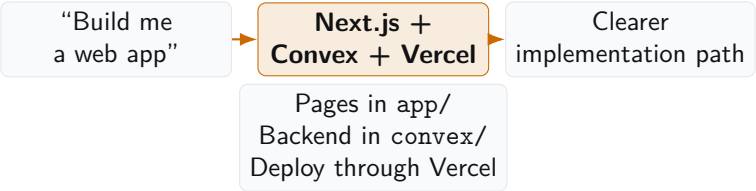
- Frontend: what users see and click
- Backend/database: where the app stores and changes data
- Hosting: how the app gets onto the internet

Our Stack



Why this stack? Familiar terrain for Codex, clean integrations, and easy enough for beginners to inspect.

The Stack Is a Map for Codex



You are not mastering the whole stack here. You are giving Codex constraints and learning enough of the map to steer it.

Why Next.js

- Frontend framework: pages, routes, components, UI
- Common enough that Codex has lots of examples
- Natural fit with Vercel deployment
- Beginner recognition: app/ is where pages and routes live

Alternatives exist; this is the low-friction path for the course.

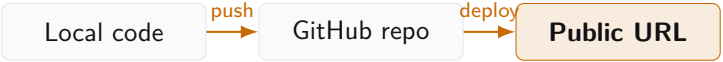
Why Convex

- Database and backend functions in TypeScript
- Less SQL/migration friction for a first agent-built app
- Dashboard makes the data inspectable
- Good fit for Codex because backend code lives near app code

For Creator Carousel Studio: drafts, brand kits, saved ideas, generated copy, and export history.

Docs: Convex CLI docs

Why Vercel



Vercel is the bridge from “Codex built it on my machine” to “I can open a real website.”

Docs: Vercel CLI docs

What You Need to Recognize

You see	Think
`app/`	frontend pages and routes
`components/`	reusable UI pieces
`convex/`	schema, queries, mutations, actions
Vercel dashboard	deployment and public URLs

Codex can explain any file, but this map makes the explanations land.

Before Building: Accounts And CLI

- Node/npm installed: `node -version` and `npm -version`
- GitHub authenticated: `gh auth status`
- Convex account ready: `npx convex dev` can log in and create a dev deployment
- Vercel account connected to GitHub
- Vercel CLI installed: `npm i -g vercel`; verify with `vercel -version`

You are learning enough of the map to steer Codex, not becoming a stack expert today.

Enough Stack Knowledge To Steer

Layer	Recognize	Ask Codex
Next.js	`app/`, routes, components	explain the page flow
Convex	`convex/`, schema, mutations	trace data reads/writes
Vercel	deployment logs, public URL	diagnose deploy failures

The stack is a steering map. Codex can fill in details as you go.

What We Just Gave Codex

- Node/npm: run the app locally
- Convex: initialize a backend and inspect data
- GitHub: store and version the code
- Vercel: deploy to a public URL
- Stack constraints: fewer choices for Codex to guess

Setup is not busywork; it prepares the environment where Codex can succeed.

SECTION 21

Worktrees

Why Worktrees Exist

Goal: editor shell, image specs, brand-kit model, and export preview can each have their own workspace when they are independent.

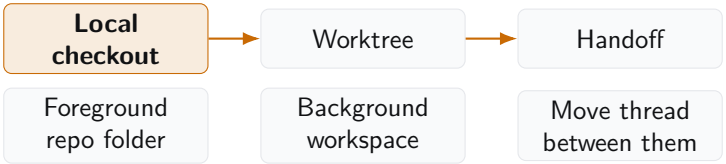
Branches vs Worktrees

Branch isolates commits	**Worktree isolates workspace**
A line of history	**A physical checkout**

Codex implication: a worktree can have its own thread, files, commands, and dev-server context.

Docs: Git worktree docs

Codex's Worktree Model



Git only allows a branch to be checked out in one worktree at a time, so Handoff matters.

Docs: Terminology and Local vs Worktree

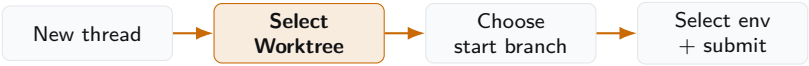
Where Worktrees Live

- Codex-managed worktrees live under `$CODEX_HOME/worktrees`
- They are real folders with real repo files
- Dependencies and build caches can take disk space
- Codex keeps recent managed worktrees and can clean up older ones
- Permanent worktrees can be created from the project menu

Translation: worktrees are lightweight in concept, but not imaginary.

Docs: How Codex manages worktrees and cleanup

How They Work in the UI



Detached HEAD

Codex can start from a commit without immediately attaching the work to a named branch. Create a branch when the changes are ready to become a real PR candidate.

Docs: Getting started

Worktrees Need Environments

- Worktrees may be missing dependencies, env files, generated files, or local state
- Codex local environments define setup scripts for new worktrees
- Actions define repeatable commands like run, test, or build

Docs: Local environment setup scripts and Actions

Manual Environment Setup in Codex

In the UI

Project settings
Local environment
Setup script
Actions

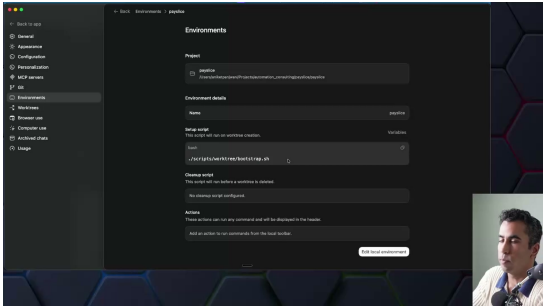
In the repo

Stored under `.codex`
Safe to commit
Shared setup contract

For Creator Carousel Studio, this is the difference between "Codex has a copy of the files" and "Codex has a checkout it can actually run."

Docs: Local environments

Environment Settings Are Part Of The Workflow



The setup script is the contract: install deps, create expected env placeholders, run project checks, and print the next commands Codex should use.

What A Worktree Does Not Copy

- `.env` files and secrets
- `node_modules`, virtualenvs, build caches
- Generated files that are not committed
- Local databases and running services
- Open dev servers and terminal state

Therefore: commit shared setup files and bootstrap the environment before expecting good agent work.

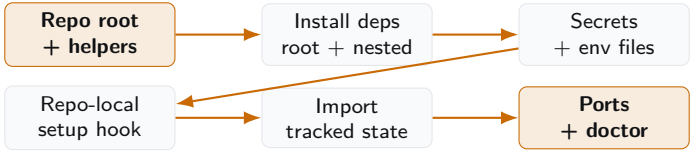
My Bootstrap Pattern

```
version = 1
name = "creator-carousel-studio"

[setup]
script = "./scripts/worktree/bootstrap.sh"
```

The bootstrap script owns dependencies, env scaffolding, Beads import, prerequisite checks, port checks, and doctor output.

Example `bootstrap.sh` Structure



Goal: one idempotent command turns a fresh worktree into a checkout Codex can actually build, test, and inspect.

Local vs Worktree

Local

Foreground checkout.
Best for inspection, your usual IDE, and the main running app.

Worktree

Background workspace.
Best for scoped independent tasks Codex can work on separately.

Use Handoff when the thread needs to move between foreground and background.

Docs: Handoff to Local

Worktrees and Pull Requests

Worktrees do not replace branches or PRs. They give each branch its own workspace while the work is being created.

Docs: Working on the worktree

What Can Go Wrong

- Two worktrees edit the same files
- One PR changes the foundation underneath another PR
- A branch is already checked out in a different worktree
- Build caches and dependencies use real disk space

```
fatal: 'feature/a' is already used by worktree at '<WORKTREE_PATH>'
```

Docs: Branch limitations

How to Handle Conflicts

- Start from an up-to-date main branch
- Keep each worktree tied to one issue
- Avoid assigning the same files to parallel tracks
- Merge the lowest-risk foundational PR first
- Update remaining branches from main after merges
- Ask Codex to resolve conflicts, rerun checks, and explain changes

Conflicts are not failure; they are a signal that two tracks touched the same design surface.

The Worktree Rule

Separate issue	Separate branch	Separate thread	**Separate acceptance**

Ideally: mostly separate files

If two changes need to be designed together, keep them together.

When Not To Parallelize

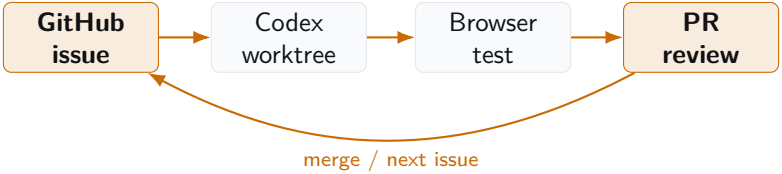
- One feature defines the design language another feature depends on.
- Two branches need to edit the same canvas/editor files.
- You still do not know what the product should be.
- The acceptance criteria require one integrated taste review.

Sequence first, parallelize later.

SECTION 22

Build Creator Carousel Studio

Capstone Loop: Issue → Worktree → Browser → PR



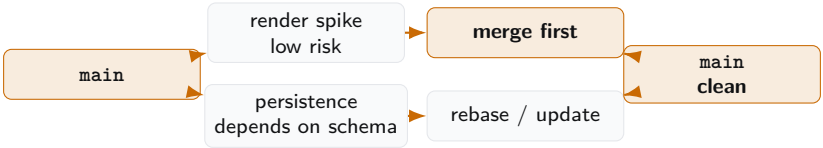
- Inspect the scaffold in the in-app browser.
- Decide whether it is good enough to checkpoint.
- Open a PR, merge it, and watch issue/PR state change.

Make Worktrees Reproducible

- Start Convex and Next.js together, not as two mystery terminals.
- Add environment/bootstrap files so future worktrees can run cleanly.
- Teach Codex the expected commands: install, dev, test, build, doctor.
- Commit the setup contract before launching more parallel work.

A worktree that cannot run is not parallel leverage; it is a folder with hope attached.

Parallel Branches Still Need A Merge Plan



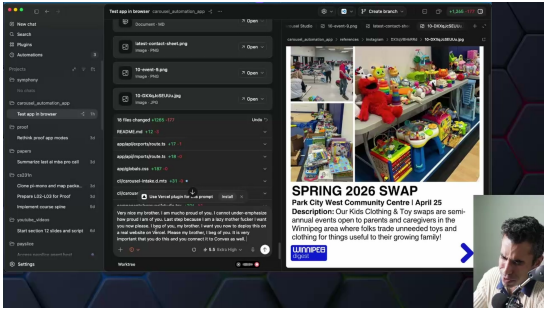
Ask Codex: "Given these PRs, what order should I merge them and why?"

When To Let Codex Run Long

- Good: prototype app, local throwaway surface, reference images, issue queue, easy browser verification.
- Risky: security, payments, auth, migrations, production data, or ambiguous product taste.
- Always end with inspection: browser test, screenshots, diff review, checks, and follow-up issues.

The point is not to disappear. The point is to supervise at the right granularity.

Reference Images As Product Spec



- Paste reference carousel screenshots into Codex.
- Ask what visual decisions matter: spacing, type hierarchy, rhythm, export dimensions.
- Let Codex implement, then paste screenshots back as critique.
- Use browser tests and exported PNGs as proof, not vibes.

SECTION 23

Use the Companion Skill

App + Skill: Two Interfaces To One Workflow

Pattern: app for inspection; skill/CLI for repeatable intake and updates.

Use The Companion Workflow To Update The App

- Feed Codex source material: notes, event lists, transcripts, or references.
- Skill turns the material into structured carousel data.
- App shows the result as editable slides.
- Browser preview catches text overflow, weak hierarchy, bad crops, and export problems.
- Repeat until the live app contains usable content.

This is why "app vs skill" was the wrong question. The strongest workflow uses both.

Companion Skill Checklist

- One clear trigger description
- One structured output format the app can ingest
- Validation or review step before writing data
- Browser verification after the data appears in the app
- Production/dev target made explicit so Codex updates the right place

Skill mistake to avoid: letting it become a second hidden app with no visual review surface.

SECTION 24

Deploy, Smoke Test, Wrap

From Local Prototype To Live Product Surface

- Deploy the web app to Vercel.
- Connect the live app to Convex.
- Open the live URL and smoke test the real workflow.
- Update live content through the companion workflow.
- Record follow-up issues for production hardening.

Live Smoke Test Checklist

- Can the deployed page load from a clean browser?
- Does it read/write the intended Convex deployment?
- Can you create or update carousel content?
- Do previews and exports work from the live URL?
- Are obvious secrets, test data, and broken links absent?

Deployment is not the finish line. The live URL is another test surface.

Production Hardening Is A Follow-Up Track

- Auth and route protection
- Data ownership and user separation
- Error monitoring and logs
- Rate limits and abuse controls
- Backup/export story
- Documentation for setup and operations

Course boundary: ship the working surface; track hardening as issues.

Shipping Checklist For Agent-Built Apps

- Reference assets and product criteria are explicit
- Environment can be bootstrapped in a fresh checkout
- Issues are small, linked to the plan, and independently verifiable
- Worktree branches merge in a deliberate order
- Browser tests and screenshots prove the UI works
- PR diffs are reviewed before merge
- Live app is smoke tested after deploy

What You Can Do After This Course

- Build useful internal tools for your own workflows.
- Prototype client or side-project apps faster.
- Turn repeatable personal procedures into skills.
- Use GitHub issues, PRs, and worktrees to make agent work inspectable.
- Decide when a workflow should remain local, become a skill, or become an app.

Start with the simplest durable workflow. Add structure only when the work earns it.

Capstone Links

Public Deck Bonus

- Creator Carousel Studio (source code): linked in description
- OpenAI Codex app docs: developers.openai.com/codex/app
- Codex app worktrees: developers.openai.com/codex/app/worktrees
- Codex app automations: developers.openai.com/codex/app/automations
- Codex app review: developers.openai.com/codex/app/review
- Convex docs: docs.convex.dev
- Vercel CLI docs: vercel.com/docs/cli
- GitHub CLI: cli.github.com

Continue Learning

Public Deck Bonus

- Join the AI MBA Skool community: skool.com/the-ai-mba
- AI MBA Pro: weekly live calls, advanced builds, and direct access as you ship with Codex
- Compound Engineering plugin: github.com/EveryInc/compound-engineering-plugin
- OpenAI skills docs: developers.openai.com/codex/skills
- Agentskills spec: agentskills.io
- Destructive Command Guard: github.com/Dicklesworthstone/destructive_command_guard
- Wispr Flow (voice dictation): wisprflow.ai
- Windows WSL setup: youtube.com/watch?v=Geq6RCYTSFA